Version Control

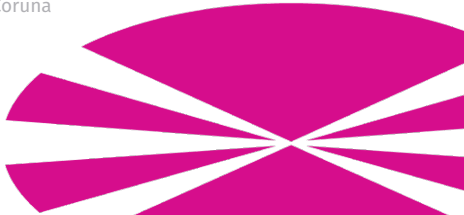
Ferramentas de Desenvolvemento

Javier Parapar López

Grao en Enxeñaría Informática · Facultade de Informática · Universidade da Coruña

Curso 2026/2027

GEI GRADO EN ENXEÑARÍA INFORMÁTICA



Where we are going

1. Why version control
2. What Git stores
3. Getting started
4. The everyday cycle
5. Working with others
6. Branching
7. Code review

SECTION 1

Why version control

1

Two things called SCM

Software Configuration Management

The discipline: tracking and controlling change in a software system.

Revision control · baselines and branching · team collaboration · defect tracking · configuration control and status accounting · build management · environment management.

Two things called SCM

Software Configuration Management

The discipline: tracking and controlling change in a software system.

Revision control · baselines and branching · team collaboration · defect tracking · configuration control and status accounting · build management · environment management.

Source Code Management

The tool at its core, and the subject of today. What it gives a team:

collaboration without stepping on each other · the complete **history** · **release notes** that are actually accurate · documentation and tests versioned *with* the code · **notifications** when something changes.

Two things called SCM

Software Configuration Management

The discipline: tracking and controlling change in a software system.

Revision control · baselines and branching · team collaboration · defect tracking · configuration control and status accounting · build management · environment management.

Source Code Management

The tool at its core, and the subject of today. What it gives a team:

collaboration without stepping on each other · the complete **history** · **release notes** that are actually accurate · documentation and tests versioned *with* the code · **notifications** when something changes.

Almost every other tool in this course reads from here. The issue tracker links to commits (*Lesson 3*), the build runs on a checkout (*Lesson 4*), the pipeline is triggered by a push (*Lesson 5*), and the analysis reports per branch (*Lesson 6*).

Two things called SCM

Software Configuration Management

The discipline: tracking and controlling change in a software system.

Revision control · baselines and branching · team collaboration · defect tracking · configuration control and status accounting · build management · environment management.

Source Code Management

The tool at its core, and the subject of today. What it gives a team:

collaboration without stepping on each other · the complete **history** · **release notes** that are actually accurate · documentation and tests versioned *with* the code · **notifications** when something changes.

Almost every other tool in this course reads from here. The issue tracker links to commits (*Lesson 3*), the build runs on a checkout (*Lesson 4*), the pipeline is triggered by a push (*Lesson 5*), and the analysis reports per branch (*Lesson 6*).

So this is not the lesson about one more tool. It is the lesson about the thing the rest of them are attached to.

Centralised, then distributed

- **Centralised** — CVS, Subversion, Perforce. One server holds the history; your machine holds a working copy. No server, no history: no `log`, no `diff`, no commit
- **Distributed** — Git, Mercurial, Bazaar, BitKeeper. Every clone is a **complete repository**, history included. You commit, branch, search and bisect **offline**, and push when you want to share

Centralised, then distributed

- **Centralised** — CVS, Subversion, Perforce. One server holds the history; your machine holds a working copy. No server, no history: no `log`, no `diff`, no commit
- **Distributed** — Git, Mercurial, Bazaar, BitKeeper. Every clone is a **complete repository**, history included. You commit, branch, search and bisect **offline**, and push when you want to share

Why the difference matters

It is not about working on a train. It is that **committing stops being a public act**: you can commit ten times while you think, and share once when it is worth reading.

That is what makes small commits — and therefore reviewable diffs — practical at all.

Centralised, then distributed

- **Centralised** — CVS, Subversion, Perforce. One server holds the history; your machine holds a working copy. No server, no history: no `log`, no `diff`, no commit
- **Distributed** — Git, Mercurial, Bazaar, BitKeeper. Every clone is a **complete repository**, history included. You commit, branch, search and bisect **offline**, and push when you want to share

Why the difference matters

It is not about working on a train. It is that **committing stops being a public act**: you can commit ten times while you think, and share once when it is worth reading.

That is what makes small commits — and therefore reviewable diffs — practical at all.

You will meet Subversion in some companies, and the ideas transfer. But Git is the default of the industry, and it is what the rest of this course assumes.

The habits, before the commands

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits
- **Say why, and link the issue.** The *what* is in the diff; the message carries the intent and the issue key (*Lesson 3*)

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits
- **Say why, and link the issue.** The *what* is in the diff; the message carries the intent and the issue key (*Lesson 3*)
- **Commit what you meant to commit.** Name the files. `git add .` is how secrets and `target/` end up in history

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits
- **Say why, and link the issue.** The *what* is in the diff; the message carries the intent and the issue key (*Lesson 3*)
- **Commit what you meant to commit.** Name the files. `git add .` is how secrets and `target/` end up in history
- **Pull often, push often.** Both reduce the size of the conflict you are eventually going to have

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits
- **Say why, and link the issue.** The *what* is in the diff; the message carries the intent and the issue key (*Lesson 3*)
- **Commit what you meant to commit.** Name the files. `git add .` is how secrets and `target/` end up in history
- **Pull often, push often.** Both reduce the size of the conflict you are eventually going to have
- **Agree on formatting.** Git compares **lines**: one reformatting editor turns a two-line change into a 400-line diff (*Lesson 1*, `.editorconfig`)

The habits, before the commands

- **One commit, one purpose** — fully implemented. If the message needs an “and”, it is two commits
- **Say why, and link the issue.** The *what* is in the diff; the message carries the intent and the issue key (*Lesson 3*)
- **Commit what you meant to commit.** Name the files. `git add .` is how secrets and `target/` end up in history
- **Pull often, push often.** Both reduce the size of the conflict you are eventually going to have
- **Agree on formatting.** Git compares **lines**: one reformatting editor turns a two-line change into a 400-line diff (*Lesson 1*, `.editorconfig`)
- **Never commit generated files**, and know what your merge tool does before the day you need it

None of this is about Git. It is about leaving a history that somebody — probably you, in March — can read.

SECTION 2

What Git stores

2

Git

- A **distributed** version control system, written in 2005 for the Linux kernel when its BitKeeper licence was withdrawn — a project with thousands of contributors and no central authority to trust
- Which explains every design decision: **speed**, a **simple design**, first-class support for **non-linear development** (thousands of parallel branches), fully **distributed**, and able to handle very large projects
- It is also why the interface is the way it is. Git was built as a set of sharp tools for people who already understood the model — the friendly commands came later



Git

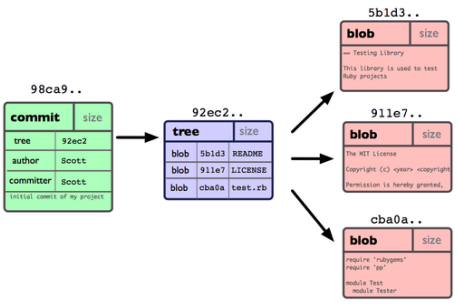
- A **distributed** version control system, written in 2005 for the Linux kernel when its BitKeeper licence was withdrawn — a project with thousands of contributors and no central authority to trust
- Which explains every design decision: **speed**, a **simple design**, first-class support for **non-linear development** (thousands of parallel branches), fully **distributed**, and able to handle very large projects
- It is also why the interface is the way it is. Git was built as a set of sharp tools for people who already understood the model — the friendly commands came later



So the fastest way to stop fighting it is to spend ten minutes on what it stores, which is the next slide. Almost every confusing Git command becomes obvious once the model is in your head.

Snapshots, not differences

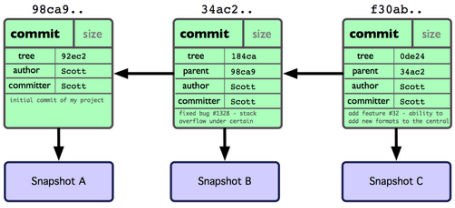
- Most older systems store a file plus a **list of changes** to it. Git stores, for every commit, a **snapshot of the whole tree** — files that did not change are simply referenced again
- Three kinds of object: a blob is the content of a file, a tree is a directory listing of blobs and trees, and a commit points to one tree plus its **parent commit**, the author and the message
- Every object is named by the **SHA-1 of its content**. Same content, same name — so history is a chain in which nothing can be changed without changing every name after it
- A branch is a 41-byte file containing one commit hash, and HEAD is a pointer to the branch you are on



one commit

Snapshots, not differences

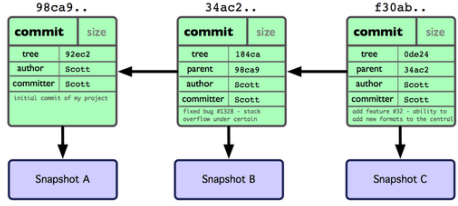
- Most older systems store a file plus a **list of changes** to it. Git stores, for every commit, a **snapshot of the whole tree** — files that did not change are simply referenced again
- Three kinds of object: a blob is the content of a file, a tree is a directory listing of blobs and trees, and a commit points to one tree plus its **parent commit**, the author and the message
- Every object is named by the **SHA-1 of its content**. Same content, same name — so history is a chain in which nothing can be changed without changing every name after it
- A branch is a 41-byte file containing one commit hash, and HEAD is a pointer to the branch you are on



and its parents

Snapshots, not differences

- Most older systems store a file plus a **list of changes** to it. Git stores, for every commit, a **snapshot of the whole tree** — files that did not change are simply referenced again
- Three kinds of object: a blob is the content of a file, a tree is a directory listing of blobs and trees, and a commit points to one tree plus its **parent commit**, the author and the message
- Every object is named by the **SHA-1 of its content**. Same content, same name — so history is a chain in which nothing can be changed without changing every name after it
- A branch is a 41-byte file containing one commit hash, and HEAD is a pointer to the branch you are on



and its parents

Consequences to keep: branching is cheap because it writes 41 bytes; history is a graph, not a list; and rewriting history changes identities — which is why rebasing what you have pushed is rude.

SECTION 3

Getting started

3

Installing and identifying yourself

Install: `apt install git` · `dnf install git` · `brew install git` · `git-scm.com` on Windows.

Then tell it who you are — this ends up in every commit you ever make, and it is what the issue tracker and the pipeline will use to blame you (kindly):

```
git config --global user.name  "Javier Parapar"
git config --global user.email javier.parapar@udc.es
git config --global core.editor nano
git config --global merge.tool  meld
git config --list
```

Settings live in three layers — `–system`, `–global` (you), and the repository's own `.git/config` — and the most specific one wins.

Installing and identifying yourself

Install: `apt install git` · `dnf install git` · `brew install git` · `git-scm.com` on Windows.

Then tell it who you are — this ends up in every commit you ever make, and it is what the issue tracker and the pipeline will use to blame you (kindly):

```
git config --global user.name  "Javier Parapar"
git config --global user.email javier.parapar@udc.es
git config --global core.editor nano
git config --global merge.tool  meld
git config --list
```

Settings live in three layers — `–system`, `–global` (you), and the repository's own `.git/config` — and the most specific one wins.

Use your real university address. A commit signed `pepe@localhost` is a commit nobody can attribute, and the individual follow-up of this course is read from the repository history.

Starting a repository — and getting help

- `git init` — turn this directory into a repository. It creates `.git/`, and that directory **is** the repository
- `git clone <url>` — get a copy of an existing one, history included:
  ```
  git clone git@git.fic.udc.es:photoalbum
  git clone <url> otherfolder
  ```
- After cloning, every file is **unmodified**: your copy and the remote agree
- `git status` — the command you will type most often. When in doubt, run it: it tells you where you are and what it thinks you are doing

Starting a repository — and getting help

- `git init` — turn this directory into a repository. It creates `.git/`, and that directory **is** the repository
- `git clone <url>` — get a copy of an existing one, history included:
  ```
  git clone git@git.fic.udc.es:photoalbum
  git clone <url> otherfolder
  ```
- After cloning, every file is **unmodified**: your copy and the remote agree
- `git status` — the command you will type most often. When in doubt, run it: it tells you where you are and what it thinks you are doing

The four ways to get help

```
git help <command>
git <command> –help
man git-<command>
git-scm.com/docs/git-<command>
```

All four show the same page. Git's manual is unusually good, and unusually unread.

Starting a repository — and getting help

- `git init` — turn this directory into a repository. It creates `.git/`, and that directory **is** the repository
- `git clone <url>` — get a copy of an existing one, history included:
  `git clone git@git.fic.udc.es:photoalbum`
  `git clone <url> otherfolder`
- After cloning, every file is **unmodified**: your copy and the remote agree
- `git status` — the command you will type most often. When in doubt, run it: it tells you where you are and what it thinks you are doing

The four ways to get help

```
git help <command>
git <command> –help
man git-<command>
git-scm.com/docs/git-<command>
```

All four show the same page. Git's manual is unusually good, and unusually unread.

Everything Git knows is inside `.git/`. Delete it and you have an ordinary folder; copy it and you have copied the whole history.

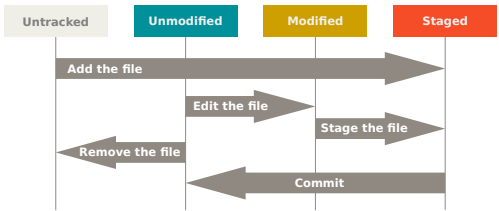
SECTION 4

The everyday cycle



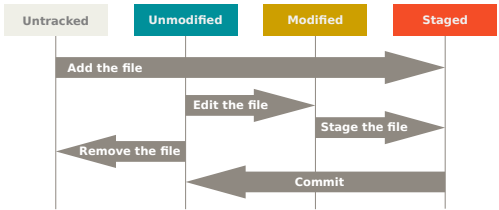
Three places a file can be

- **Tracked** files were part of the last commit. At any moment each one is **unmodified**, **modified** or **staged**
- **Untracked** files are in your directory and Git is ignoring them — they are not in the last commit and not staged
- The middle box is the one other systems do not have: the staging area (the *index*) is where you **compose** the next commit, file by file, hunk by hunk
- `git status` answers, for every file, which of these it is



Three places a file can be

- **Tracked** files were part of the last commit. At any moment each one is **unmodified**, **modified** or **staged**
- **Untracked** files are in your directory and Git is ignoring them — they are not in the last commit and not staged
- The middle box is the one other systems do not have: the staging area (the *index*) is where you **compose** the next commit, file by file, hunk by hunk
- `git status` answers, for every file, which of these it is



The staging area is what makes "one commit, one purpose" possible when you have just fixed two things at once. It is not bureaucracy — it is the tool for telling a clean story about messy work.

Composing the next commit

COMMAND	EFFECT
`git add <file>`	stage an untracked file, or the changes to a tracked one
`git add -p`	stage **part** of a file, hunk by hunk
`git rm <file>`	remove it from the folder *and* from the repository
`git rm –cached <file>`	stop tracking it, keep it on disk
`git mv <a> <b>`	exactly `mv`, then `git rm a`, then `git add b`
`git clean -n` / `-f`	list / delete untracked files

Composing the next commit

COMMAND	EFFECT
`git add <file>`	stage an untracked file, or the changes to a tracked one
`git add -p`	stage **part** of a file, hunk by hunk
`git rm <file>`	remove it from the folder *and* from the repository
`git rm –cached <file>`	stop tracking it, keep it on disk
`git mv <a> <b>`	exactly `mv`, then `git rm a`, then `git add b`
`git clean -n` / `-f`	list / delete untracked files

The subtlety that bites everybody once: `git add` stages the file **as it is at that instant**. Edit it again before committing and the commit contains the *first* version — `status` will show it as both staged and modified.

Composing the next commit

COMMAND	EFFECT
`git add <file>`	stage an untracked file, or the changes to a tracked one
`git add -p`	stage **part** of a file, hunk by hunk
`git rm <file>`	remove it from the folder *and* from the repository
`git rm –cached <file>`	stop tracking it, keep it on disk
`git mv <a> <b>`	exactly `mv`, then `git rm a`, then `git add b`
`git clean -n` / `-f`	list / delete untracked files

The subtlety that bites everybody once: `git add` stages the file **as it is at that instant**. Edit it again before committing and the commit contains the *first* version — `status` will show it as both staged and modified.

`git clean` has no undo: it deletes files Git has never seen, so nothing can bring them back. Run it with `-n` first, always.

What must never be committed

`.gitignore` lists what Git should not even offer to track. It is committed with the project, because it is a property of the **project**, not of your machine:

```
# comment line is ignored
*.class            # no compiled classes
build/             # ignore the whole build directory
target/            # Maven output (Lesson 4)
doc/*.txt          # text files from doc
.env               # local secrets: never, ever
```

Three families: **generated** files (they come back from the build), **local** files (your IDE's private state, your paths), and **secrets** (keys, passwords, tokens).

What must never be committed

`.gitignore` lists what Git should not even offer to track. It is committed with the project, because it is a property of the **project**, not of your machine:

```
# comment line is ignored
*.class            # no compiled classes
build/             # ignore the whole build directory
target/            # Maven output (Lesson 4)
doc/*.txt          # text files from doc
.env               # local secrets: never, ever
```

Three families: **generated** files (they come back from the build), **local** files (your IDE's private state, your paths), and **secrets** (keys, passwords, tokens).

A secret that reaches a repository is published, even if you delete it in the next commit: it lives in the history, in every clone. The fix is not a new commit — it is rotating the credential.

Seeing what you changed

- `git diff` — what you changed and have **not** staged
- `git diff --staged` — what is staged, and therefore what the next commit will contain
- `git diff <rev> [<path>]` — against any revision, optionally limited to a path
- Read the header: `-` is the old side, `+++` the new one, and `@@` gives the line numbers of the hunk

```
diff --git a/README b/README
index 228f38a..28ac49c 100755
--- a/README
+++ b/README
@@ -7,5 +7,5 @@ Jenkins
 Sonar
 Redmine
 CodePro Analytix
-
+Eclipse
```

Seeing what you changed

- `git diff` — what you changed and have **not** staged
- `git diff –staged` — what is staged, and therefore what the next commit will contain
- `git diff <rev> [<path>]` — against any revision, optionally limited to a path
- Read the header: – is the old side, `+++` the new one, and @@ gives the line numbers of the hunk

```
diff --git a/README b/README
index 228f38a..28ac49c 100755
--- a/README
+++ b/README
@@ -7,5 +7,5 @@ Jenkins
 Sonar
 Redmine
 CodePro Analytix
-
+Eclipse
```

Get into the habit of reading `git diff –staged` before every commit. It is the cheapest code review there is, and it catches the debugging line you forgot to delete.

Committing

- `git commit` — opens your editor with a template and the list of what is about to be committed
- `git commit -m "..."` — message on the command line
- `git commit -a -m "..."` — stage every **tracked** file and commit. Convenient, and the reason people commit things they did not mean to
- A commit records: the tree, the parent, the author, the date and the message. All of it, forever

```
# Please enter the commit message for your
# changes. Lines starting with '#' will be
# ignored, and an empty message aborts the
# commit.
#
# Committer: Javier Parapar <jparapar@udc.es>
#
# On branch main
# Changes to be committed:
#       modified:   README
#
```

Committing

- `git commit` — opens your editor with a template and the list of what is about to be committed
- `git commit -m "..."` — message on the command line
- `git commit -a -m "..."` — stage every **tracked** file and commit. Convenient, and the reason people commit things they did not mean to
- A commit records: the tree, the parent, the author, the date and the message. All of it, forever

```
# Please enter the commit message for your
# changes. Lines starting with '#' will be
# ignored, and an empty message aborts the
# commit.
#
# Committer: Javier Parapar <jparapar@udc.es>
#
# On branch main
# Changes to be committed:
#       modified:   README
#
```

Read that list in the template before you save. It is Git telling you exactly what you are about to sign your name to.

A commit message is documentation

The diff already says *what* changed. The message exists to say **why** — and it is the only part of your work that somebody will read six months from now without asking you first.

The shape

A short **subject line** (about 50 characters, imperative: *fix the timeout*, not *fixed* or *fixes*), a blank line, then a body that explains the reasoning and the alternatives you rejected.

And a footer the tracker understands: `refs #42` to link it, `closes #42` to close it (*Lesson 3*).

A commit message is documentation

The diff already says *what* changed. The message exists to say **why** — and it is the only part of your work that somebody will read six months from now without asking you first.

The shape

A short **subject line** (about 50 characters, imperative: *fix the timeout*, not *fixed* or *fixes*), a blank line, then a body that explains the reasoning and the alternatives you rejected.

And a footer the tracker understands: `refs #42` to link it, `closes #42` to close it (*Lesson 3*).

Conventional Commits

A convention many teams enforce:
`<type>[scope]: <description>`

`fix:` → a **patch** release · `feat:` → a **minor** one · `BREAKING CHANGE:` (or `feat!:`) → a **major** one. Also `docs`, `test`, `refactor`, `ci`, `build`, `perf`, `chore`.

A commit message is documentation

The diff already says *what* changed. The message exists to say **why** — and it is the only part of your work that somebody will read six months from now without asking you first.

The shape

A short **subject line** (about 50 characters, imperative: *fix the timeout*, not *fixed* or *fixes*), a blank line, then a body that explains the reasoning and the alternatives you rejected.

And a footer the tracker understands: `refs #42` to link it, `closes #42` to close it (*Lesson 3*).

Conventional Commits

A convention many teams enforce:
`<type>[scope]: <description>`

`fix:` → a **patch** release · `feat:` → a **minor** one · `BREAKING CHANGE:` (or `feat!:`) → a **major** one. Also `docs`, `test`, `refactor`, `ci`, `build`, `perf`, `chore`.

Why a machine-readable message pays: the pipeline can derive the **version number** and generate the **changelog** from the history, with no human writing release notes at midnight (*Lessons 4 and 5*).

Reading the history

COMMAND	SHOWS
`git log`	commits, newest first: hash, author, date, message
`git log -n 5`	the last five
`git log -p`	each commit **with its diff**
`git log –stat`	how many lines changed, per file
`git log –graph –oneline`	the branch topology, in ASCII
`git log –pretty=format:"..."`	any format you like, for scripts
`git log -S "someString"`	the commits that **added or removed** that string
`gitk` / the editor's view	the same thing, graphically (*Lesson 1*)

Reading the history

COMMAND	SHOWS
`git log`	commits, newest first: hash, author, date, message
`git log -n 5`	the last five
`git log -p`	each commit **with its diff**
`git log –stat`	how many lines changed, per file
`git log –graph –oneline`	the branch topology, in ASCII
`git log –pretty=format:"..."`	any format you like, for scripts
`git log -S "someString"`	the commits that **added or removed** that string
`gitk` / the editor's view	the same thing, graphically (*Lesson 1*)

`git log -S` is the one nobody knows and everybody needs: it searches the **history** rather than the working tree, and answers "when did this constant appear, and who put it there?"

Reading the history

COMMAND	SHOWS
`git log`	commits, newest first: hash, author, date, message
`git log -n 5`	the last five
`git log -p`	each commit **with its diff**
`git log –stat`	how many lines changed, per file
`git log –graph –oneline`	the branch topology, in ASCII
`git log –pretty=format:"..."`	any format you like, for scripts
`git log -S "someString"`	the commits that **added or removed** that string
`gitk` / the editor's view	the same thing, graphically (*Lesson 1*)

`git log -S` is the one nobody knows and everybody needs: it searches the **history** rather than the working tree, and answers "when did this constant appear, and who put it there?"

All of this is only as good as your commits. A history of "changes", "changes 2" and "final" cannot be searched, blamed or bisected — which is the next slide.

Finding the commit that broke it

It worked three weeks ago and it is broken now, with 400 commits in between. git bisect does a **binary search** over history: you say *good* and *bad*, it checks out the middle, you test, it halves again.

```
git bisect start
git bisect bad                  # the current commit is broken
git bisect good v1.2            # this tag was fine
   ... git checks out the middle commit; you test and answer ...
git bisect good | git bisect bad
git bisect reset                # back to where you were
```

Finding the commit that broke it

It worked three weeks ago and it is broken now, with 400 commits in between. git bisect does a **binary search** over history: you say *good* and *bad*, it checks out the middle, you test, it halves again.

```
git bisect start
git bisect bad                  # the current commit is broken
git bisect good v1.2            # this tag was fine
   ... git checks out the middle commit; you test and answer ...
git bisect good | git bisect bad
git bisect reset                # back to where you were
```

400 commits is nine tests, not 400. And if the test can be scripted you do not even answer: `git bisect run ./test.sh` — exit status 0 means good — and Git finds the culprit by itself.

Finding the commit that broke it

It worked three weeks ago and it is broken now, with 400 commits in between. git bisect does a **binary search** over history: you say *good* and *bad*, it checks out the middle, you test, it halves again.

```
git bisect start
git bisect bad                 # the current commit is broken
git bisect good v1.2           # this tag was fine
   ... git checks out the middle commit; you test and answer ...
git bisect good | git bisect bad
git bisect reset               # back to where you were
```

400 commits is nine tests, not 400. And if the test can be scripted you do not even answer: `git bisect run ./test.sh` — exit status 0 means good — and Git finds the culprit by itself.

This is why the habits at the start of the lesson pay off: bisect only works on a history of small commits that each build.

The discipline of week one is what buys you this in week ten.

Undoing, without rewriting what others have

- Forgot a file in the last commit? Add it and **amend**:
  ```
  git add forgottenfile
  git commit –amend
  ```
- Undo a commit that is already **public**:
  ```
  git revert <rev>
  ```
  — it creates a **new** commit that reverses it, so nobody's history changes. Needs a clean working tree
- Unstage something you added by accident:
  ```
  git reset HEAD <file>
  ```
- Throw away your changes to one file:
  ```
  git checkout – <file>
  ```

Undoing, without rewriting what others have

- Forgot a file in the last commit? Add it and **amend**:
  `git add forgottenfile`
  `git commit –amend`
- Undo a commit that is already **public**:
  `git revert <rev>`
  — it creates a **new** commit that reverses it, so nobody's history changes. Needs a clean working tree
- Unstage something you added by accident:
  `git reset HEAD <file>`
- Throw away your changes to one file:
  `git checkout – <file>`

The dividing line

Amend rewrites the last commit; **revert** adds a new one.

So: amend and reset while the work is **still yours**; once it is pushed and somebody may have pulled it, the only polite undo is **revert**.

Undoing, without rewriting what others have

- Forgot a file in the last commit? Add it and **amend**:
  `git add forgottenfile`
  `git commit –amend`
- Undo a commit that is already **public**:
  `git revert <rev>`
  — it creates a **new** commit that reverses it, so nobody's history changes. Needs a clean working tree
- Unstage something you added by accident:
  `git reset HEAD <file>`
- Throw away your changes to one file:
  `git checkout – <file>`

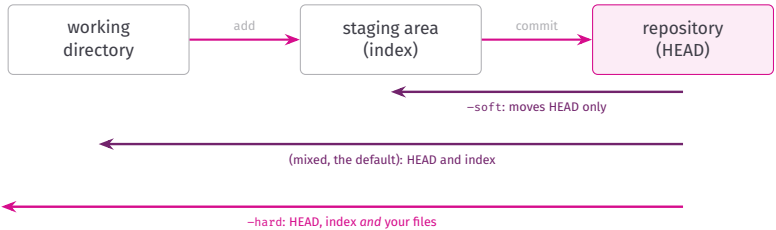
The dividing line

Amend rewrites the last commit; **revert** adds a new one.

So: amend and reset while the work is **still yours**; once it is pushed and somebody may have pulled it, the only polite undo is **revert**.

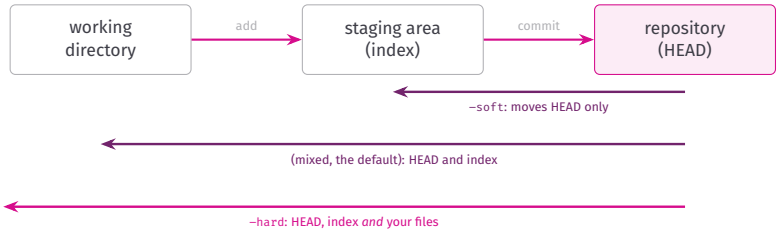
`git checkout – <file>` deletes work with no confirmation and no undo: that version was never committed, so Git never had it. It is the command to be afraid of.

The three resets, in one picture



`git reset [mode] HEAD~1` moves the branch pointer back one commit. What changes is **how far the undo reaches**: with `–soft` your changes stay staged, ready to re-commit; by default they stay in your files but unstaged; with `–hard` they are **gone**.

The three resets, in one picture



`git reset [mode] HEAD~1` moves the branch pointer back one commit. What changes is **how far the undo reaches**: with `–soft` your changes stay staged, ready to re-commit; by default they stay in your files but unstaged; with `–hard` they are **gone**.

Rule of thumb: `–soft` to rewrite the message or split a commit, default to rethink what goes in, `–hard` only when you are certain — and never on a branch somebody else has.

SECTION 5

Working with others

5

Remotes

A remote is a named URL of another copy of this repository. `origin` is just the conventional name for the one you cloned from — there is nothing special about it, and you can have several.

```
git remote -v                       origin  git@git.fic.udc.es:photoalbum.git (fetch)
git remote add upstream <url>       origin  git@git.fic.udc.es:photoalbum.git (push)
git remote show origin              # branches, tracking, ahead/behind
git remote rename old new           git remote rm name
```

fetch brings their commits into your repository and changes *nothing* in your files. **pull** is fetch **and** merge — it touches your working tree, which is why it can fail or conflict.

Remotes

A remote is a named URL of another copy of this repository. `origin` is just the conventional name for the one you cloned from — there is nothing special about it, and you can have several.

```
git remote -v                       origin  git@git.fic.udc.es:photoalbum.git (fetch)
git remote add upstream <url>       origin  git@git.fic.udc.es:photoalbum.git (push)
git remote show origin              # branches, tracking, ahead/behind
git remote rename old new           git remote rm name
```

fetch brings their commits into your repository and changes *nothing* in your files. **pull** is fetch **and** merge — it touches your working tree, which is why it can fail or conflict.

Uncommitted work in the way of a pull? Put it aside and take it back afterwards: `git stash`, `git pull`, `git stash pop`.

Tags: naming a commit a human can remember

A branch moves; a tag does not. It is a permanent name for one commit — which is what a **release** is.

```
git tag -a v0.1alpha -m 'initial alpha version'     # annotated: author, date, message
git tag                                             # list them
git show v0.1alpha                                  # the commit behind the name
git tag -a v0.1alpha -m '...' 8192e29               # tag an older commit
git push origin v0.1alpha        git push origin --tags
```

Note the last line: **tags are not pushed by a normal push**. If nobody else can see your tag, that is why.

Tags: naming a commit a human can remember

A branch moves; a tag does not. It is a permanent name for one commit — which is what a **release** is.

```
git tag -a v0.1alpha -m 'initial alpha version'     # annotated: author, date, message
git tag                                             # list them
git show v0.1alpha                                  # the commit behind the name
git tag -a v0.1alpha -m '...' 8192e29               # tag an older commit
git push origin v0.1alpha        git push origin --tags
```

Note the last line: **tags are not pushed by a normal push**. If nobody else can see your tag, that is why.

Tags are where several lessons meet: they are what Lesson 4 builds a versioned artefact from, what Lesson 5 deploys, and what a hotfix branch is cut from when production breaks.

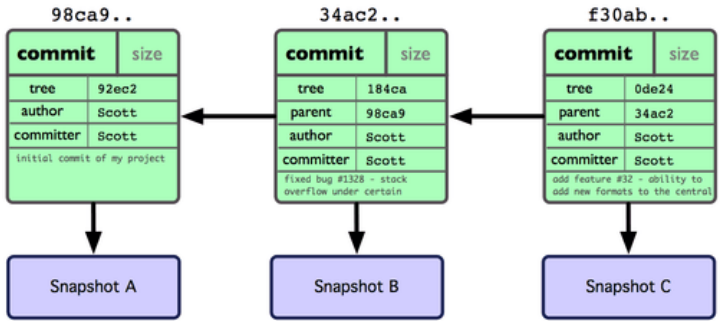
SECTION 6

Branching

6

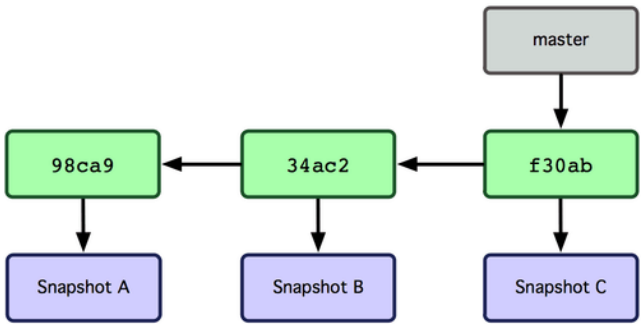
A branch is a pointer

A commit points to a **tree** and to its **parent**. That is the history.



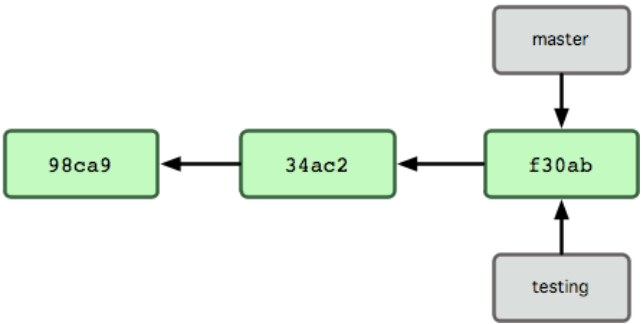
A branch is a pointer

A **branch** is just a movable pointer to one of those commits.



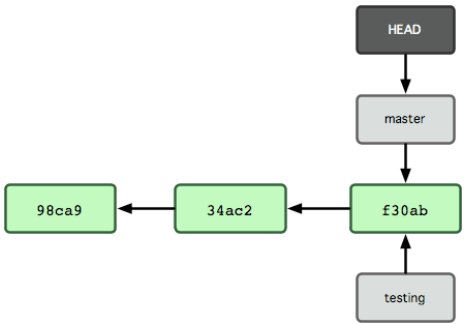
A branch is a pointer

`git branch testing` — create a second pointer to the same commit.



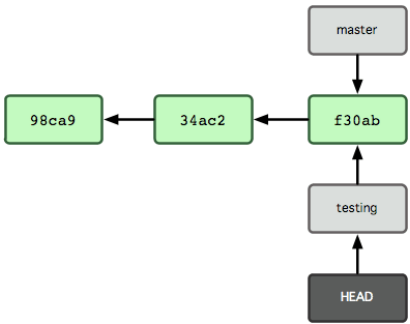
A branch is a pointer

HEAD is the pointer that says **which branch you are on**.



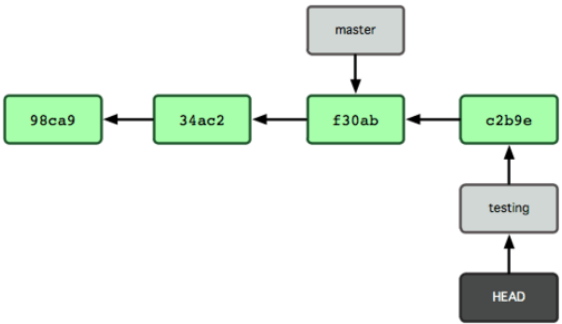
A branch is a pointer

`git checkout testing` — move HEAD, and your files with it.



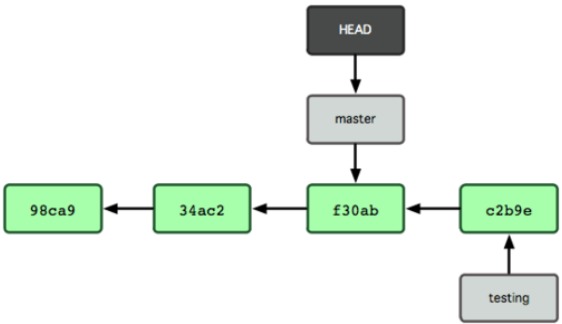
A branch is a pointer

Commit here and `testing` moves forward; `master` does not.



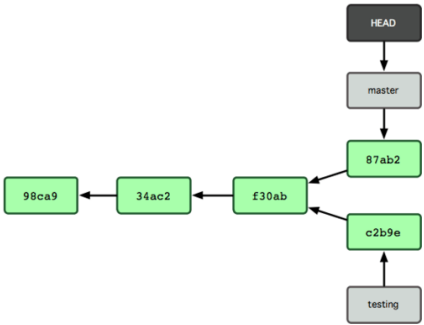
A branch is a pointer

`git checkout master` — back, and the working tree is rewritten.



A branch is a pointer

Commit again and the two lines have **diverged**. This is the normal state.



Remember what a branch physically **is**: a file with a 40-character hash in it. Creating one is instantaneous, deleting one costs nothing, and that is why Git encourages branching where older systems punished it.

Branches that live on the server

```
git push origin localname:remotename      # publish a branch
git checkout -b localname origin/remotename    # track a remote branch locally
git checkout --track origin/remotename         # the same thing, shorter
git push origin :remotename               # delete the remote branch
git branch -d localname                   # delete it locally (-D to force)
```

`origin/main` is not a branch you work on: it is Git's **memory of where that branch was on the server**, as of the last time you talked to it. That is why `git fetch` can change it, and why `git status` then tells you that you are "2 commits behind".

Branches that live on the server

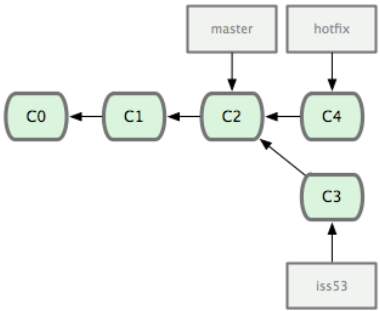
```
git push origin localname:remotename      # publish a branch
git checkout -b localname origin/remotename    # track a remote branch locally
git checkout --track origin/remotename         # the same thing, shorter
git push origin :remotename               # delete the remote branch
git branch -d localname                   # delete it locally (-D to force)
```

`origin/main` is not a branch you work on: it is Git's **memory of where that branch was on the server**, as of the last time you talked to it. That is why `git fetch` can change it, and why `git status` then tells you that you are "2 commits behind".

Branches on the server are cheap too, and a branch whose name carries the issue key — `42-fix-login-timeout` — tells everybody what it is for without asking (Lesson 3).

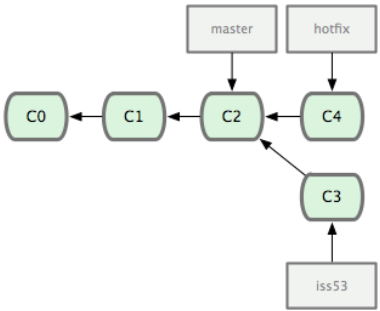
Merging

We want the work in `hotfix` to become part of `master`.



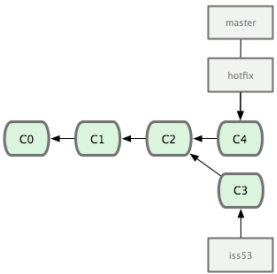
Merging

`git checkout master` — you always merge **into** the branch you are on.



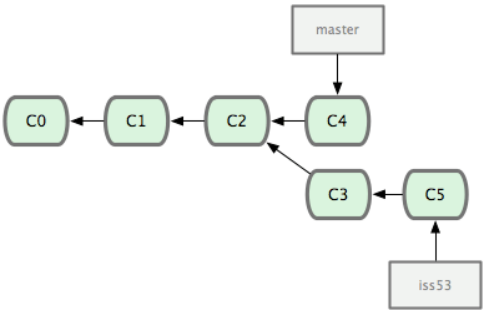
Merging

`git merge hotfix` — *Fast forward*: nothing diverged, so Git just moves the pointer.



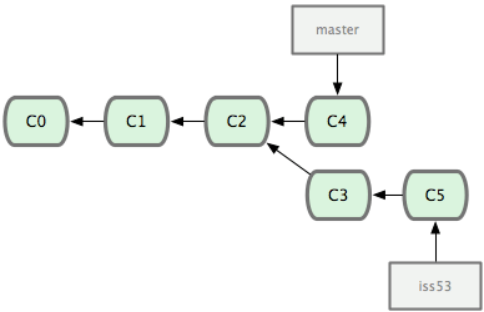
Merging

`git branch -d hotfix` — the commits are in `master` now; the pointer is not needed.



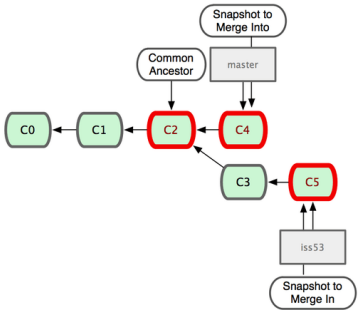
Merging

Now the harder case: `iss53` and `master` have **both** moved.



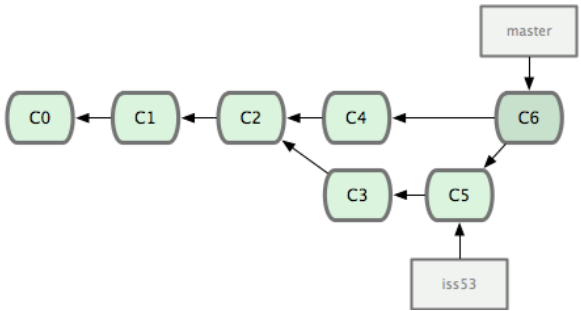
Merging

`git merge iss53` — Git finds the **common ancestor** and does a three-way merge.



Merging

It writes a new commit with **two parents**. That is what a merge commit is.



A **fast forward** is not a merge at all, just a pointer move. Some teams forbid it (`git merge –no-ff`) precisely so that every feature leaves a visible merge commit in the history.

When Git cannot decide

```
git merge iss53
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

A conflict is not an error: it is Git **refusing to guess** when two people changed the same lines. The procedure is always the same:

1. `git status` — it lists exactly which files are unmerged
2. Open them and resolve: the markers show **your side** and **theirs**. Or use `git mergetool` for a three-pane view
3. `git add` the resolved files — staging **is** how you say “resolved”
4. `git commit` to finish the merge

When Git cannot decide

```
git merge iss53
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

A conflict is not an error: it is Git **refusing to guess** when two people changed the same lines. The procedure is always the same:

1. `git status` — it lists exactly which files are unmerged
2. Open them and resolve: the markers show **your side** and **theirs**. Or use `git mergetool` for a three-pane view
3. `git add` the resolved files — staging **is** how you say “resolved”
4. `git commit` to finish the merge

The real lesson: conflicts scale with how long a branch has lived. Two days of divergence is a conversation; two months is a project. This is the argument for the next slide.

Four ways teams use branches

STRATEGY	ONE BRANCH PER…	WATCH OUT FOR
Release	version being prepared	two lines to maintain; fix applied to only one
Feature	feature under development	long-lived branches that drift from the trunk
Task	issue in the tracker	nothing much — the name carries the issue key
Hotfix	emergency, cut from the tag	remembering to merge it *back* everywhere

They are not exclusive: most teams use task branches day to day, cut a release branch when they must support a version in the wild, and a hotfix branch when production is on fire.

Four ways teams use branches

STRATEGY	ONE BRANCH PER…	WATCH OUT FOR
Release	version being prepared	two lines to maintain; fix applied to only one
Feature	feature under development	long-lived branches that drift from the trunk
Task	issue in the tracker	nothing much — the name carries the issue key
Hotfix	emergency, cut from the tag	remembering to merge it *back* everywhere

They are not exclusive: most teams use task branches day to day, cut a release branch when they must support a version in the wild, and a hotfix branch when production is on fire.

Feature flags are the companion idea: merge the code early, but keep the feature **switched off** in production until it is ready. The code is integrated and tested continuously; only the visibility is deferred.

Four ways teams use branches

STRATEGY	ONE BRANCH PER…	WATCH OUT FOR
Release	version being prepared	two lines to maintain; fix applied to only one
Feature	feature under development	long-lived branches that drift from the trunk
Task	issue in the tracker	nothing much — the name carries the issue key
Hotfix	emergency, cut from the tag	remembering to merge it *back* everywhere

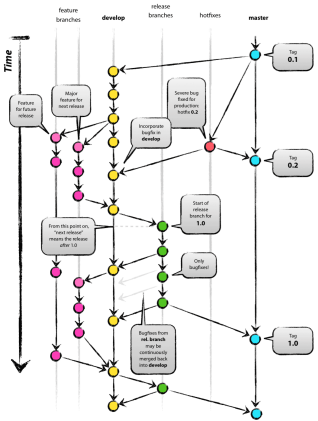
They are not exclusive: most teams use task branches day to day, cut a release branch when they must support a version in the wild, and a hotfix branch when production is on fire.

Feature flags are the companion idea: merge the code early, but keep the feature **switched off** in production until it is ready. The code is integrated and tested continuously; only the visibility is deferred.

Which is the trick that makes the modern answer possible — because it removes the reason to keep a branch alive for weeks.

Git-flow

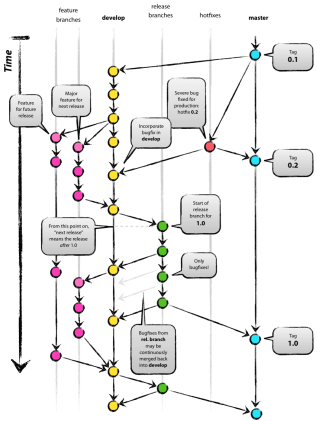
- Work happens in **feature branches**, cut from **develop** and merged back when ready
- When a release approaches, a **release branch** is cut from develop, tested and fixed in place
- On release it is merged into **master** *and* back into develop, and tagged
- **Hotfix branches** come off master for emergencies, and also go back to both



Git-flow

- Work happens in **feature branches**, cut from **develop** and merged back when ready
- When a release approaches, a **release branch** is cut from develop, tested and fixed in place
- On release it is merged into **master** *and* back into develop, and tagged
- **Hotfix branches** come off master for emergencies, and also go back to both

Five kinds of branch and a lot of bookkeeping — which buys real support for multiple published versions. It is what this course asked for until last year.



…and what its author said in 2020

Vincent Driessen published git-flow in 2010, for software that is **explicitly versioned**. Ten years later he added a note at the top of that very article:

> *"If your team is doing continuous delivery of software, I would suggest to adopt a much simpler workflow instead of trying to shoehorn git-flow into your team."*

…and what its author said in 2020

Vincent Driessen published git-flow in 2010, for software that is **explicitly versioned**. Ten years later he added a note at the top of that very article:

> *"If your team is doing continuous delivery of software, I would suggest to adopt a much simpler workflow instead of trying to shoehorn git-flow into your team."*

His reasoning: web applications are **continuously delivered**, not rolled back, and you do not support several versions at once. Git-flow was designed for the other world — the one where a customer is still running 1.4 and expects a fix for it.

…and what its author said in 2020

Vincent Driessen published git-flow in 2010, for software that is **explicitly versioned**. Ten years later he added a note at the top of that very article:

> *"If your team is doing continuous delivery of software, I would suggest to adopt a much simpler workflow instead of trying to shoehorn git-flow into your team."*

His reasoning: web applications are **continuously delivered**, not rolled back, and you do not support several versions at once. Git-flow was designed for the other world — the one where a customer is still running 1.4 and expects a fix for it.

This is a good habit to take from the course: when a practice is recommended, ask what it was designed for. Very few of them are wrong; most of them are answers to a question you may not have.

What most teams do now: one trunk, short branches

Trunk-based development: everybody works off one branch, in branches that live **hours or a day** and are merged back as soon as they are green. Releases are cut from the trunk; unfinished work hides behind flags.

What DORA measured

Teams perform better — in speed *and* stability — when they:

have **three or fewer** active branches · merge to trunk **at least once a day** · have **no code freezes** and no integration phases.

What most teams do now: one trunk, short branches

Trunk-based development: everybody works off one branch, in branches that live **hours or a day** and are merged back as soon as they are green. Releases are cut from the trunk; unfinished work hides behind flags.

What DORA measured

Teams perform better — in speed *and* stability — when they:

have **three or fewer** active branches · merge to trunk **at least once a day** · have **no code freezes** and no integration phases.

Why it works

A conflict is proportional to divergence, and divergence is proportional to **time**. Merging daily does not make conflicts easier to resolve — it makes them **small**.

It is also what makes continuous integration (*Lesson 5*) mean anything.

What most teams do now: one trunk, short branches

Trunk-based development: everybody works off one branch, in branches that live **hours or a day** and are merged back as soon as they are green. Releases are cut from the trunk; unfinished work hides behind flags.

What DORA measured

Teams perform better — in speed *and* stability — when they:

have **three or fewer** active branches · merge to trunk **at least once a day** · have **no code freezes** and no integration phases.

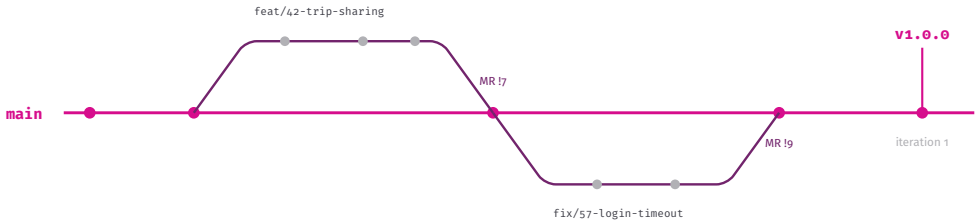
Why it works

A conflict is proportional to divergence, and divergence is proportional to **time**. Merging daily does not make conflicts easier to resolve — it makes them **small**.

It is also what makes continuous integration (*Lesson 5*) mean anything.

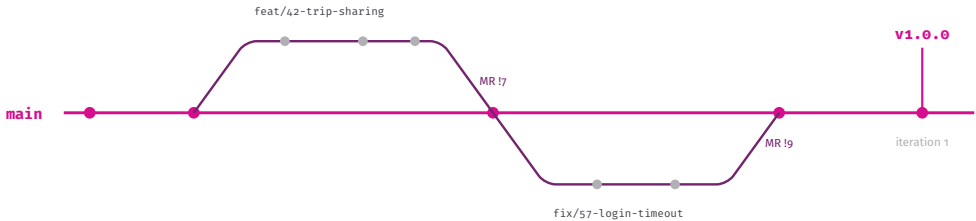
This is the same finding as the report in the introduction: the bottleneck is rarely the tool, it is how long work sits unintegrated — and it is why the next slide is the strategy this course now requires.

The strategy for this course



One trunk, short branches, a tag per iteration. This is **GitLab Flow**, it is what the industry does, and it is what the pipeline of *Lesson 5* expects. From this year it **replaces git-flow** in the assignment.

The strategy for this course



One trunk, short branches, a tag per iteration. This is **GitLab Flow**, it is what the industry does, and it is what the pipeline of *Lesson 5* expects. From this year it **replaces git-flow** in the assignment.

Read the picture as a rule: nothing reaches `main` except through a merge request, and `main` is always in a state you could deploy.

The rules, for the assignment

The trunk

- `main` is **protected**: nobody pushes to it, ever
- `main` is **always deployable**: its pipeline is green
- Every iteration is closed with an **annotated tag**: `v1.0.0`, `v2.0.0`, ...

The branches

- **One issue, one branch**, named after the issue
- A branch lives **two days at most**. Longer than that and it is too big a task, not a longer branch
- Branches are **not deleted** after merging: we read them when marking

The merge request

- Opened **early**, as a draft if it is not ready — it is where the work is discussed, not where it is announced
- Merged only with a **green pipeline** and **one approval** from another member of the team
- **Squashed** on merge: one issue becomes one commit in `main`, with a readable message
- Every member **reviews** and **is reviewed**: it is not a role one of you takes for the whole course

The rules, for the assignment

The trunk

- `main` is **protected**: nobody pushes to it, ever
- `main` is **always deployable**: its pipeline is green
- Every iteration is closed with an **annotated tag**: `v1.0.0`, `v2.0.0`, ...

The branches

- **One issue, one branch**, named after the issue
- A branch lives **two days at most**. Longer than that and it is too big a task, not a longer branch
- Branches are **not deleted** after merging: we read them when marking

The merge request

- Opened **early**, as a draft if it is not ready — it is where the work is discussed, not where it is announced
- Merged only with a **green pipeline** and **one approval** from another member of the team
- **Squashed** on merge: one issue becomes one commit in `main`, with a readable message
- Every member **reviews** and **is reviewed**: it is not a role one of you takes for the whole course

All of it is visible in the repository, and all of it is marked: branch names, branch lifetimes, who reviewed whom, and whether `main` was ever red. The history is the evidence of how you worked.

Branch names, and why they are a convention

`<type>/<issue>-<short-description>`: the type vocabulary of the commit messages, the issue number from the tracker of *Lesson 3*, and two or three words in lower case, hyphenated.

TYPE	FOR	EXAMPLE
`feat`	a new capability	`feat/42-trip-sharing`
`fix`	a defect in something that exists	`fix/57-login-timeout`
`refactor`	structure changes, behaviour does not	`refactor/61-extract-service`
`test`	tests added or repaired	`test/64-expenses-coverage`
`docs`	documentation only	`docs/70-api-readme`
`chore`	build, dependencies, configuration	`chore/72-bump-spring-boot`

Branch names, and why they are a convention

`<type>/<issue>-<short-description>`: the type vocabulary of the commit messages, the issue number from the tracker of *Lesson 3*, and two or three words in lower case, hyphenated.

TYPE	FOR	EXAMPLE
`feat`	a new capability	`feat/42-trip-sharing`
`fix`	a defect in something that exists	`fix/57-login-timeout`
`refactor`	structure changes, behaviour does not	`refactor/61-extract-service`
`test`	tests added or repaired	`test/64-expenses-coverage`
`docs`	documentation only	`docs/70-api-readme`
`chore`	build, dependencies, configuration	`chore/72-bump-spring-boot`

Why bother: the list of branches becomes a **readable status of the team** — what is in flight, of what kind, for which issue — and the number links both ways, task to code and code to task.

Branch names, and why they are a convention

`<type>/<issue>-<short-description>`: the type vocabulary of the commit messages, the issue number from the tracker of *Lesson 3*, and two or three words in lower case, hyphenated.

TYPE	FOR	EXAMPLE
`feat`	a new capability	`feat/42-trip-sharing`
`fix`	a defect in something that exists	`fix/57-login-timeout`
`refactor`	structure changes, behaviour does not	`refactor/61-extract-service`
`test`	tests added or repaired	`test/64-expenses-coverage`
`docs`	documentation only	`docs/70-api-readme`
`chore`	build, dependencies, configuration	`chore/72-bump-spring-boot`

Why bother: the list of branches becomes a **readable status of the team** — what is in flight, of what kind, for which issue — and the number links both ways, task to code and code to task.

Same idea in the commits: `feat(trips): share a trip` in the subject, `refs #42` or `closes #42` in the footer, and the tracker links the two by itself.

The life of one change, command by command

```
git checkout main && git pull            # start from what everybody has
git checkout -b feat/42-trip-sharing     # one issue, one branch

  ... work, in small commits ...
git add src/main/java/...Trip.java       # name what goes in
git commit -m "feat(trips): share a trip with a friend

A trip can be shared with another user now. refs #42"

git push -u origin feat/42-trip-sharing  # the first push creates it
```

Then, in GitLab: **open the merge request** (the push prints the link), let the **pipeline** run, ask a teammate for **review**, fix what they raise **as new commits**, and **squash-merge** once it is green and approved.

The life of one change, command by command

```
git checkout main && git pull            # start from what everybody has
git checkout -b feat/42-trip-sharing     # one issue, one branch

  ... work, in small commits ...
git add src/main/java/...Trip.java       # name what goes in
git commit -m "feat(trips): share a trip with a friend

A trip can be shared with another user now. refs #42"

git push -u origin feat/42-trip-sharing  # the first push creates it
```

Then, in GitLab: **open the merge request** (the push prints the link), let the **pipeline** run, ask a teammate for **review**, fix what they raise **as new commits**, and **squash-merge** once it is green and approved.

And afterwards, `git checkout main && git pull` **before starting the next branch**: every branch starts from an up-to-date trunk, which is what keeps conflicts small.

The life of one change, command by command

```
git checkout main && git pull            # start from what everybody has
git checkout -b feat/42-trip-sharing     # one issue, one branch

  ... work, in small commits ...
git add src/main/java/...Trip.java       # name what goes in
git commit -m "feat(trips): share a trip with a friend

A trip can be shared with another user now. refs #42"

git push -u origin feat/42-trip-sharing  # the first push creates it
```

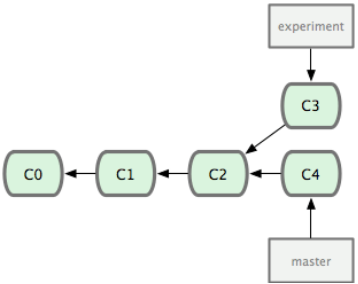
Then, in GitLab: **open the merge request** (the push prints the link), let the **pipeline** run, ask a teammate for **review**, fix what they raise **as new commits**, and **squash-merge** once it is green and approved.

And afterwards, `git checkout main && git pull` **before starting the next branch**: every branch starts from an up-to-date trunk, which is what keeps conflicts small.

At the end of the iteration, one person tags it: `git tag -a v1.0.0 -m "Iteration 1"`, then `git push origin v1.0.0`.

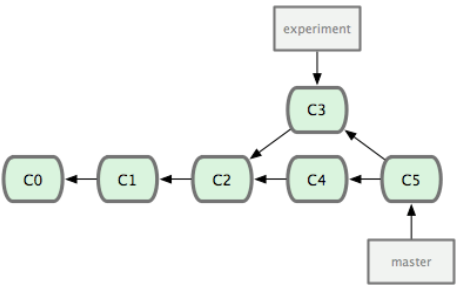
Rebasing

Two lines of work. We want them together.



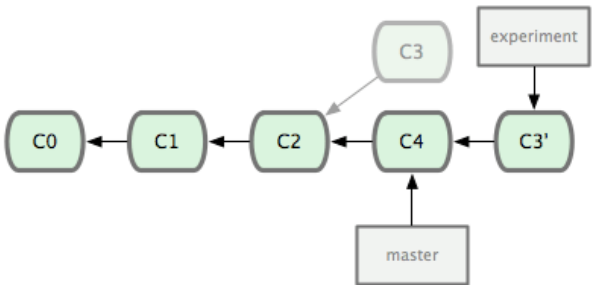
Rebasing

`merge` joins them with a commit that has two parents.



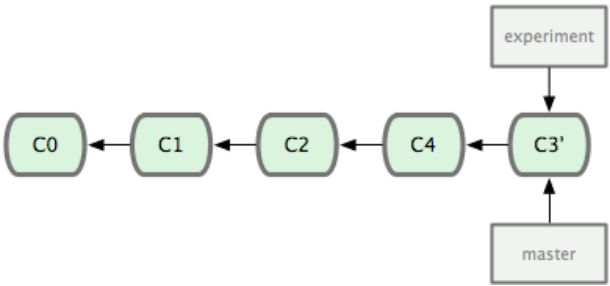
Rebasing

Rebase instead *replays* your commits on top of the other branch.



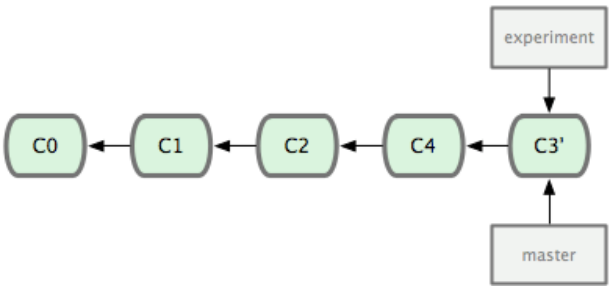
Rebasing

`git checkout experiment` then `git rebase master`.



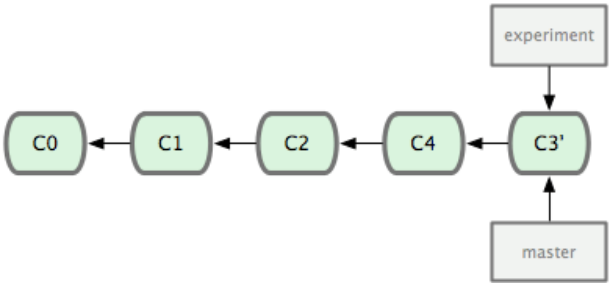
Rebasing

Now master can fast-forward: the history is a **straight line**.



Rebasing

Same code as the merge. Different history — and different commit hashes.

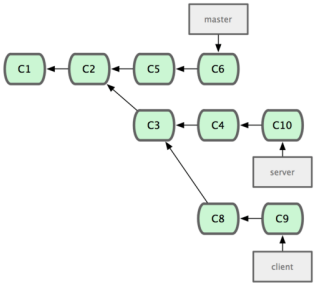


Never rebase commits you have already pushed to a shared branch.

It creates **new commits with new hashes**: everyone who pulled the old ones now has a history that no longer exists upstream.

Rebasing

A harder case: only some of these commits should move.

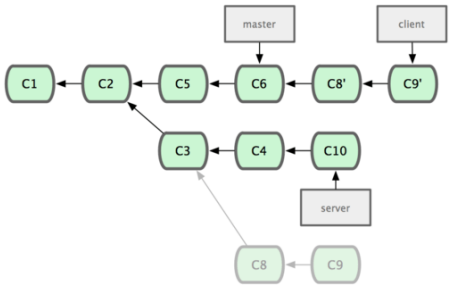


Never rebase commits you have already pushed to a shared branch.

It creates **new commits with new hashes**: everyone who pulled the old ones now has a history that no longer exists upstream.

Rebasing

`git rebase --onto master server client` — take client's own commits only.

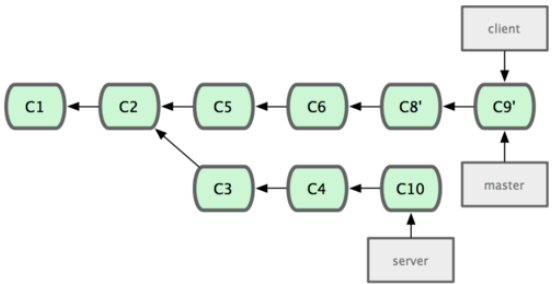


Never rebase commits you have already pushed to a shared branch.

It creates **new commits with new hashes**: everyone who pulled the old ones now has a history that no longer exists upstream.

Rebasing

Then fast-forward master and delete what is no longer needed.



Never rebase commits you have already pushed to a shared branch.

It creates **new commits with new hashes**: everyone who pulled the old ones now has a history that no longer exists upstream.

How to make everybody's life worse

- Commit files that can be **regenerated** from what is already committed
- Commit **personal configuration**, or anything secret
- Commit **large binaries** — they stay in every clone forever
- Let the repository grow without ever asking why

- `reset –hard` or `–merge` without committing or stashing first
- `checkout` on a file without realising it **discards** your changes
- `git clean` without `-n`
- `pull` or `fetch` with a branch argument you do not understand
- **Rewriting public history**

How to make everybody's life worse

- Commit files that can be **regenerated** from what is already committed
- Commit **personal configuration**, or anything secret
- Commit **large binaries** — they stay in every clone forever
- Let the repository grow without ever asking why

- `reset --hard` or `--merge` without committing or stashing first
- `checkout` on a file without realising it **discards** your changes
- `git clean` without `-n`
- `pull` or `fetch` with a branch argument you do not understand
- **Rewriting public history**

The left column costs your team time forever. The right column costs you an afternoon — except the last one, which costs everybody an afternoon.

SECTION 7

Code review

7

The merge request

A merge request (a *pull request* elsewhere) is a request to merge one branch into another — and, more usefully, the **place where that change is discussed before it happens**.

- Compare two branches, with the full diff
- **Comment on specific lines**, and resolve the threads
- Build, test and analyse **per branch**, before merging (*Lessons 5 and 6*)
- Block the merge until it is ready: approvals, green pipeline, no unresolved threads
- **Squash** on merge, to keep the trunk's history readable

The merge request

A merge request (a *pull request* elsewhere) is a request to merge one branch into another — and, more usefully, the **place where that change is discussed before it happens**.

- Compare two branches, with the full diff
- **Comment on specific lines**, and resolve the threads
- Build, test and analyse **per branch**, before merging (*Lessons 5 and 6*)
- Block the merge until it is ready: approvals, green pipeline, no unresolved threads
- **Squash** on merge, to keep the trunk's history readable

The life of one change

branch → commits → merge request → feedback → fixes, as **new commits** → pipeline green → approval → merge → deploy.

Every arrow is automated except two: the feedback and the approval. Those are the ones that need people, which is why they are the subject of this section.

The merge request

A merge request (a *pull request* elsewhere) is a request to merge one branch into another — and, more usefully, the **place where that change is discussed before it happens**.

- Compare two branches, with the full diff
- **Comment on specific lines**, and resolve the threads
- Build, test and analyse **per branch**, before merging (*Lessons 5 and 6*)
- Block the merge until it is ready: approvals, green pipeline, no unresolved threads
- **Squash** on merge, to keep the trunk's history readable

The life of one change

branch → commits → merge request → feedback → fixes, as **new commits** → pipeline green → approval → merge → deploy.

Every arrow is automated except two: the feedback and the approval. Those are the ones that need people, which is why they are the subject of this section.

And it is where the previous lessons show up as numbers: the pipeline result, the test report, the coverage, the new code smells and the licence check of your dependencies, all attached to the change that caused them.

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

- Does it make sense *read as a change*, rather than as the thing you were thinking about for two days?

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

- Does it make sense *read as a change*, rather than as the thing you were thinking about for two days?
- Is there anything in here **unrelated** to its purpose? Extract it into another branch or another issue

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

- Does it make sense *read as a change*, rather than as the thing you were thinking about for two days?
- Is there anything in here **unrelated** to its purpose? Extract it into another branch or another issue
- Did you leave **debugging code**, a commented-out block, or a `TODO` with nobody's name on it?

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

- Does it make sense *read as a change*, rather than as the thing you were thinking about for two days?
- Is there anything in here **unrelated** to its purpose? Extract it into another branch or another issue
- Did you leave **debugging code**, a commented-out block, or a `TODO` with nobody's name on it?
- Is the **pipeline green**, and the description complete? Making a reviewer discover a failing build is spending their time to save your own

Before you ask anyone to look at it

The first reviewer of your code is **you**. Read the whole diff before you push, and ask:

- Does it make sense *read as a change*, rather than as the thing you were thinking about for two days?
- Is there anything in here **unrelated** to its purpose? Extract it into another branch or another issue
- Did you leave **debugging code**, a commented-out block, or a `TODO` with nobody's name on it?
- Is the **pipeline green**, and the description complete? Making a reviewer discover a failing build is spending their time to save your own
- Did you **comment your own diff** where something needs explaining — the odd workaround, the thing you were unsure about?

And ask the right person. If the change touches an area somebody knows better than you, that is not an admission of weakness: not asking is how a solution ships with nobody confident in it.

And if you are the reviewer

- You are looking at **architecture, separation of concerns, tests, consistency and readability** — not at whether you would have named it differently
- Be **thorough**, to keep the number of rounds down; say which comments are blocking and which are preferences
- Look for what is **missing**: the error path, the edge case, the test for the bug being fixed
- Check the **security surface** — anything touching authentication, session handling, stored data or cryptography deserves a second pair of eyes
- Do not approve on a red pipeline, and do not dismiss a finding to shorten the list

And if you are the reviewer

- You are looking at **architecture, separation of concerns, tests, consistency and readability** — not at whether you would have named it differently
- Be **thorough**, to keep the number of rounds down; say which comments are blocking and which are preferences
- Look for what is **missing**: the error path, the edge case, the test for the bug being fixed
- Check the **security surface** — anything touching authentication, session handling, stored data or cryptography deserves a second pair of eyes
- Do not approve on a red pipeline, and do not dismiss a finding to shorten the list

What the evidence says

From the classic study at Cisco:

review **200–400 lines** at a time · slower than **500 lines per hour** · not more than about **60 minutes** in one sitting.

Kept inside those limits, a review finds **70–90 %** of the defects that are there. Outside them, attention collapses — and so does the number.

And if you are the reviewer

- You are looking at **architecture, separation of concerns, tests, consistency and readability** — not at whether you would have named it differently
- Be **thorough**, to keep the number of rounds down; say which comments are blocking and which are preferences
- Look for what is **missing**: the error path, the edge case, the test for the bug being fixed
- Check the **security surface** — anything touching authentication, session handling, stored data or cryptography deserves a second pair of eyes
- Do not approve on a red pipeline, and do not dismiss a finding to shorten the list

What the evidence says

From the classic study at Cisco:

review **200–400 lines** at a time · slower than **500 lines per hour** · not more than about **60 minutes** in one sitting.

Kept inside those limits, a review finds **70–90 %** of the defects that are there. Outside them, attention collapses — and so does the number.

Read that box backwards and it is an instruction for authors: a 2 000-line merge request will not be reviewed. It will be approved.

How to say it

Reviewing

- Ask, do not demand: *what do you think about naming this* `user_id`*?*
- Accept that much of this is subjective. Discuss the trade-off and settle it quickly
- Avoid “mine” and “yours”. It is the project’s code
- Never words that describe a person — *dumb*, *stupid*. Assume everyone is competent and well-meaning
- Say when you are unsure: *I’m not sure, let’s look it up*
- Avoid *always*, *never*, *endlessly*. And be careful with sarcasm in writing

Being reviewed

- The review is about the **code**. It is not about you
- Be grateful for a good catch, and say so
- Answer **every** comment, even if only to agree
- Explain the reasoning rather than defending the line: *it is like that because… would it be clearer if I renamed it?*
- Push fixes as **new commits**, so the reviewer can see what changed since their comment. Squash at the end, not before
- If a thread goes past three rounds, talk — then post a summary

How to say it

Reviewing

- Ask, do not demand: *what do you think about naming this* `user_id`*?*
- Accept that much of this is subjective. Discuss the trade-off and settle it quickly
- Avoid “mine” and “yours”. It is the project’s code
- Never words that describe a person — *dumb*, *stupid*. Assume everyone is competent and well-meaning
- Say when you are unsure: *I’m not sure, let’s look it up*
- Avoid *always*, *never*, *endlessly*. And be careful with sarcasm in writing

Being reviewed

- The review is about the **code**. It is not about you
- Be grateful for a good catch, and say so
- Answer **every** comment, even if only to agree
- Explain the reasoning rather than defending the line: *it is like that because… would it be clearer if I renamed it?*
- Push fixes as **new commits**, so the reviewer can see what changed since their comment. Squash at the end, not before
- If a thread goes past three rounds, talk — then post a summary

This is the most transferable page of the course. Everything else here has a version number; this does not.

Your turn

To read

- **Pro Git**, Chacon & Straub — free at `git-scm.com/book`. Chapters 2, 3 and 10 are this lesson
- *Version Control with Git*, Loeliger & McCullough, O'Reilly
- `git-scm.com/docs` — the reference
- Driessen, *A successful Git branching model* (2010, with the 2020 note) and DORA's page on trunk-based development

Exercise, in your practical group: break something on purpose. One of you commits a change that makes a test fail, mixed in among a few innocent commits. Another clones, uses `git bisect run` to find the culprit, and reverts it with `git revert`. Then do it again with the branch on the server, and open the merge request.

“It is easy to shoot your foot off with Git, but also easy to revert to a previous foot and merge it with your current leg.”

Jack William Bell

Version Control

Grao en Enxeñaría Informática

@jparapar · javierparapar@udc.es · www.dc.fi.udc.es/~parapar

GEI GRADO EN ENXEÑARÍA INFORMÁTICA

Appendix — handy configuration

Aliases

```
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.st status
git config --global alias.unstage 'reset HEAD
--'
git config --global alias.last 'log -1 HEAD'
```

Completion and prompt

```
source ~/.git-completion.bash
```

Tab-completion for commands, branches and remotes, and a prompt that shows the current branch. Both ship with Git.

Worth knowing

`git reflog` — everything `HEAD` has pointed at recently. It is how you recover a commit you thought you destroyed.

Git Cheat Sheet

Remember: git command --help

Global Git configuration is stored in $HOME/.gitconfig (git config --help)

Create

From existing data
```
cd ~/projects/myproject
git init
git add .
```

From existing repo
```
git clone ~/existing/repo ~/new/repo
git clone git://host.org/project.git
git clone ssh://you@host.org/proj.git
```

Show

Files changed in working directory
```
git status
```

Changes to tracked files
```
git diff
```

What changed between $ID1 and $ID2
```
git diff $id1  $id2
```

History of changes
```
git log
```

History of changes for file with diffs
```
git log -p $file $dir/ec/tory/
```

Who changed what and when in a file
```
git blame $file
```

A commit identified by $ID
```
git show $id
```

A specific file from a specific $ID
```
git show $id:$file
```

All local branches
```
git branch
```
(star '*' marks the current branch)

Cheat Sheet Notation

$id : notation used in this sheet to represent either a commit id, branch or a tag name
$file : arbitrary file name
$branch : arbitrary branch name

Concepts

Git Basics

```
master    : default development branch
origin    : default upstream repository
HEAD      : current branch
HEAD^     : parent of HEAD
HEAD~4    : the great-great grandparent of HEAD
```

Revert

Return to the last committed state
```
git reset --hard
```
⚠ you cannot undo a hard reset

Revert the last commit
```
git revert HEAD
```
Creates a new commit

Revert specific commit
```
git revert $id
```
Creates a new commit

Fix the last commit
```
git commit -a --amend
```
(after editing the broken files)

Checkout the $id version of a file
```
git checkout $id $file
```

Branch

Switch to the $id branch
```
git checkout $id
```

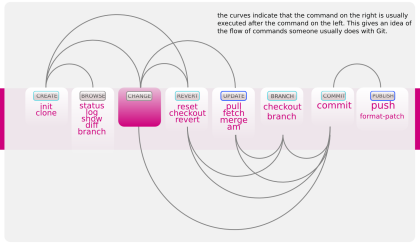
Merge branch1 into branch2
```
git checkout $branch2
git merge branch1
```

Create branch named $branch based on the HEAD
```
git branch $branch
```

Create branch $new_branch based on branch $other and switch to it
```
git checkout -b $new_branch $other
```

Delete branch $branch
```
git branch -d $branch
```

Commands Sequence



Update

Fetch latest changes from origin
```
git fetch
```
(but this does not merge them).

Pull latest changes from origin
```
git pull
```
(does a fetch followed by a merge)

Apply a patch that some sent you
```
git am -3 patch.mbox
```
(in case of a conflict, resolve and use `git am --resolved`)

Publish

Commit all your local changes
```
git commit -a
```

Prepare a patch for other developers
```
git format-patch origin
```

Push changes to origin
```
git push
```

Mark a version / milestone
```
git tag v1.0
```

Useful Commands

Finding regressions
```
git bisect start      (to start)
git bisect good $id   ($id is the last working version)
git bisect bad $id    ($id is a broken version)

git bisect bad/good   (to mark it as bad or good)
git bisect visualize  (to launch gitk and mark it)
git bisect reset      (once you're done)
```

Check for errors and cleanup repository
```
git fsck
git gc --prune
```

Search working directory for foo()
```
git grep "foo()"
```

Resolve Merge Conflicts

To view the merge conclicts
```
git diff      (complete conflict diff)
git diff --base  $file  (against base file)
git diff --ours  $file  (against your changes)
git diff --theirs $file  (against other changes)
```

To discard conflicting patch
```
git reset --hard
git rebase --skip
```

After resolving conflicts, merge with
```
git add $conflicting_file  (do for all resolved files)
git rebase --continue
```

Zack Rusin
Based on the work of
Jason Harris and
Xpirinx Cetin.